



Module 7.1

Arithmetic Building Blocks

cādence[®]

This page does not contain notes.

Objective

In this module, you will:

- Identify the basic building blocks for implementing binary arithmetic using synthesis.

Topics include:

- Addition
- Subtraction
- Implementation examples

All other arithmetic operations (multiplication, division, trigonometry) can be implemented with combinations of adders and other simple logic (e.g., shift registers).



This submodule will look at basic building blocks used by synthesis for binary arithmetic. We will look at addition, subtraction, and selected implementation examples. Note that all other arithmetic operations can be implemented with combinations of adders and simple logic, such as shift operations.

Half-Adder

Adds two binary digits producing sum and carry.

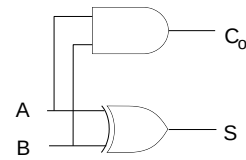
Truth Table

A	B	S	C _o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

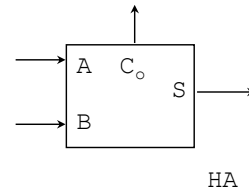
Boolean Equations

$$S = A \oplus B_i$$

$$C_o = A \cdot B$$



Gate Implementation



Schematic Symbol

Most basic building block for performing binary arithmetic.

The most basic arithmetic block is the half-adder. This adds two single inputs, A and B, to give a sum S and carry C output, as shown by the truth table. The truth table gives us the simple gate-level implementation of an AND gate for the carry generation and an XOR gate for the sum.

Full-Adder

Add carry-in to make a full adder.

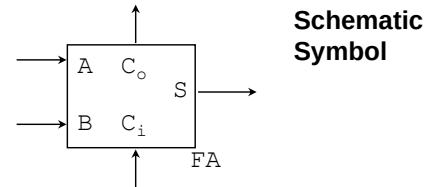
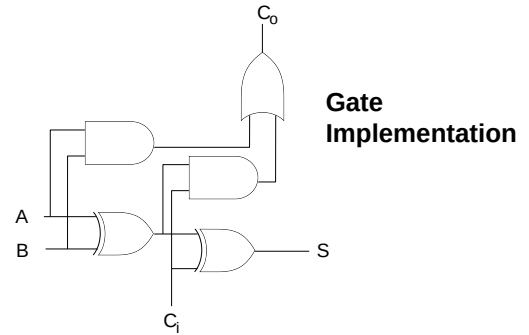
Truth Table

A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Equations

$$S = A \oplus B \oplus C_i$$

$$C_o = A \cdot B + C_i \cdot (A \oplus B)$$



Full adder required to make efficient N-bit adder.

To make a full-adder, we must expand the truth table for a third input, carry-in. From the gate implementation, we can see the full-adder is simply 2 half-adders, one to add the inputs A and B, and the other to add this result to carry-in and produce the full-adder sum. The full-adder also has an OR gate to or the carries from the two half-adders to give the full-adder carry-out.

Implementation of Addition

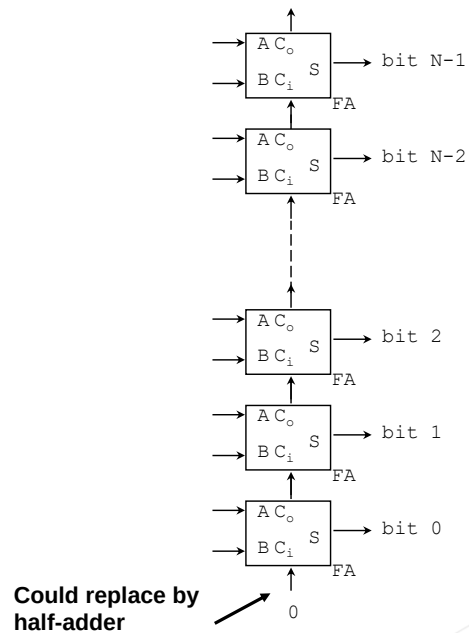
For N-bit addition, cascade N full adders.

- Ripple carry adder architecture.

Relatively slow:

- Assuming gates all have the same delay, the delay from C_i to C_o is two unit delays.
- Delay from LSB to C_o of bit N-1 is $2N$ unit delays.
 - For example, for an 8-bit adders, the delay is 16 units.

Question: How could you do a subtraction?



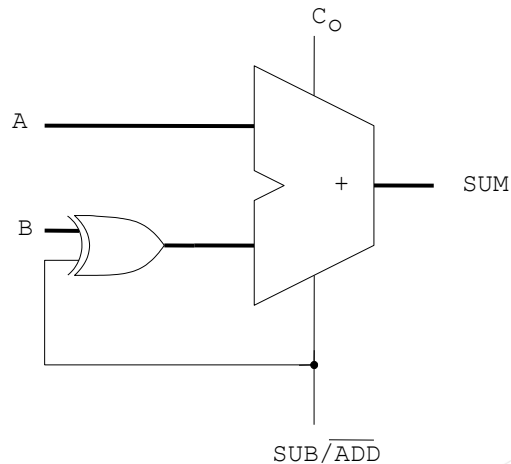
To implement N-bit addition, we can cascade N full adders. Each full-adder adds a bit of the two inputs with the carry-in from the previous bit, generates the sum, and passes the carry-out to the adder for the next bit. At the Least Significant Bit (LSB) bit 0, the carry-in is wired to 0, or we could replace the bit0 full-adder with a half-adder. This architecture is called a ripple carry adder, as the carries ripple up the adder cascade. This makes the architecture relatively slow. If we assume all gates have the same unit delay, then the propagation delay from carry-in to carry-out for each stage is two units. In the worst-case scenario of the carry from bit0 cascading all the way to bit N-1, the overall adder delay is $2N$ units. For an 8-bit adder, the delay is 16 units. This performs addition; how can we implement subtraction?

Implementation of Subtraction

Can be done using half or full-subtractor.

Can also be done by remembering that $-B = \overline{B} + 1$

Full Subtractor	B_i	A	B	D	B_o
	0	0	0	0	0
	0	0	1	1	0
	0	1	0	1	1
	0	1	1	0	0
	1	0	0	1	1
	1	0	1	0	0
	1	1	0	0	1
	1	1	1	1	1



6 © Cadence Design Systems, Inc. All rights reserved.



$$A - B = A + (B)_{2s \text{ complement}} = A + \sim B + 1$$

Implement by inverting the data into B input and set C0 to 1.

To make a combined adder/subtractor, pass the B inputs through an XOR gate, combining each bit with $\sim \text{add/sub}$ to provide controllable inversion and sourcing C0 from $\sim \text{add/sub}$.

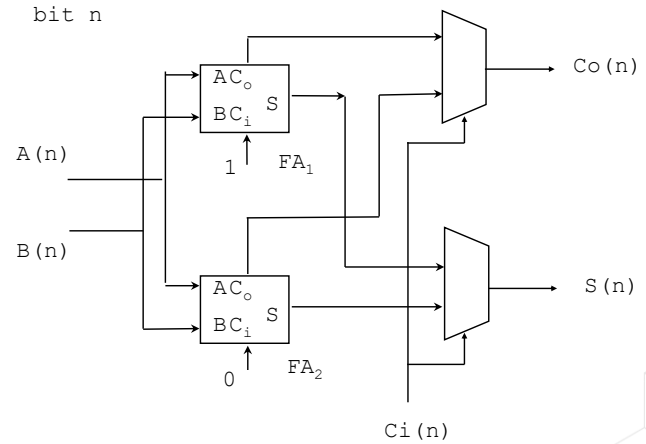
We could implement half and full subtractor blocks to implement a ripple-borrow subtractor. However, remember that in 2s complement, to make a number negative, we invert and add one. So, $A - B$ is $A + 2s$ complement of B, which is $A + \text{inversion } B \text{ plus } 1$. We can add one at the carry-in of the bit0 adder, and we could also use this bit to invert B using an XOR gate. If SUB/ADDbar is 1, we are inverting B and adding one at bit0, so the result is $A - B$. If SUB/ADDbar is 0, B is not inverted, the carry-in is 0, and the result is $A + B$. Therefore, the same architecture can be used for both addition and subtraction.

Faster Addition: Carry Select Adder (CSA)

Many different architectures to speed up addition.

For example, Carry Select Adder (CSA).

- Two ripple carry adders for each bit:
 - One hard-wired for carry in of 1
 - $Ci_1 = '1'$
 - One hard-wired for carry in of 0
 - $Ci_2 = '0'$
- Carry-in now selects between pre-computed values via multiplexers.
- Delay Ci to Co determined by the speed of the multiplexer.
- CSA architectures commonly used by specialist datapath compilers.



7

© Cadence Design Systems, Inc. All rights reserved.

cadence

Carry-Select Adder trades area for speed.

CSA architecture is quite fast.

- Carry may still ripple from LSB to MSB, but since they carry can select from pre-computed addition, speed is only restricted by delay through a multiplexor.

There are many different adder architectures with better speed than the ripple-carry adder. Here is one example, the Carry Select Adder (CSA). This architecture implements two full adders for each bit, one hardwired for a carry-in of 1 and the other for a carry-in of 0. The carry-in for a stage now selects between these pre-calculated values via a MUX. A carry may still ripple up from bit0 to bitN-1, but now the delay is determined by the speed of the MUX, which is easier to optimize. So, a CSA architecture is faster but over twice the size of a ripple carry adder. In reality, CSA may be restricted to 4-bit slices of the addition, with ripple carry used between slices. This gives a good balance between area, speed, and complexity. To access a CSA architecture in synthesis, you may need a specialist datapath synthesis tool.

Summary

In this module, you reviewed various methods of implementing basic arithmetic:

- Half adder
- Full adder
- Ripple Carry adder
- Subtraction
- Carry Select adder

There are time/area tradeoffs for different approaches.



For simple operations, the synthesis tool can infer arithmetical operations from code and select architecture from time/area constraints.

For more complex operations, the structure and architecture of the operation must be built by the designer.

In summary, there are many options for implementing basic arithmetic in hardware. We have seen how half-adder blocks can be built into full-adders for a ripple carry architecture adder, which can also be used for subtraction. We also saw an alternative adder architecture called the Carry Select Adder. Each different architecture has trade-offs for time and area.



Module 7-2

Arithmetic Implementation and Optimization

cā dence[®]

This page does not contain notes.

Objective

In this module, you will:

- Identify the issue in implementing complex binary arithmetic.

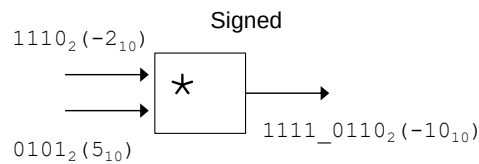
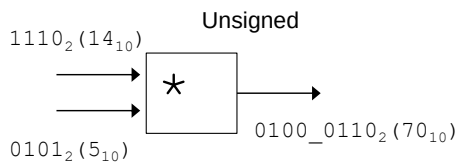
Topics include:

- Multiplication
- Division
- Approximations
- Number series



This page does not contain notes.

Multiplication and Division



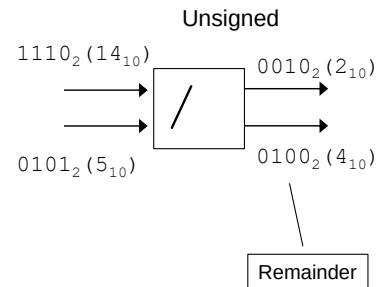
Synthesis tools can directly implement simple operations

- Addition, subtraction
- Multiplication and division by powers of 2

Other more complex operations may need to be 'designed':

- Signed and unsigned multiplication
- Division by numbers other than powers of 2

Break operation down into synthesizable blocks



Simple arithmetic operations can be directly implemented in hardware by synthesis tools. For example, addition and subtraction. Also, multiplication or division by powers of 2 are implemented with shifts.

More complex operations cannot be directly synthesized. For example, signed and unsigned multiplication and division. Also, some operations may need to be divided over several clock cycles

More complex operations may need to be designed, i.e. broken down into simpler addition, subtraction or shifts.

Simple Multiplication and Division

Multiplication and division by powers of 2 can be implemented with shift operations

- Simple re-ordering of the bus bits
- Consumes zero resources and zero time

Shift left to multiply, shift right to divide

- Assuming left-most MSB

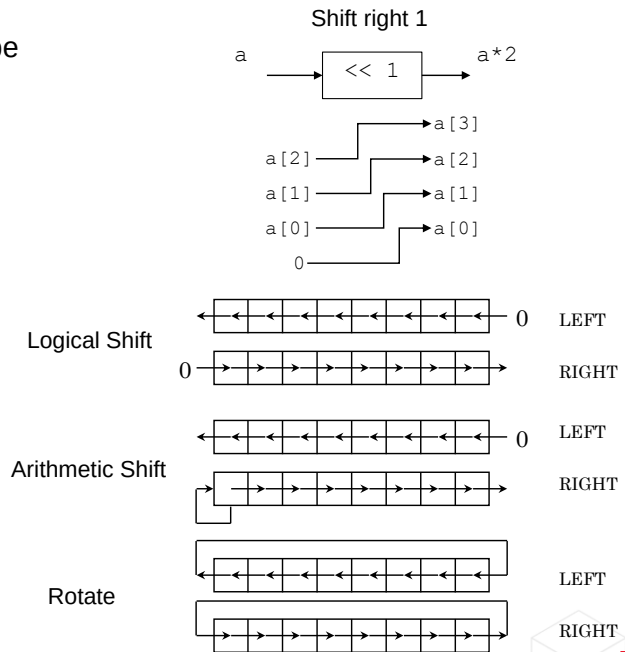
Logical shifts insert 0s

- Used for unsigned arithmetic only as they make negative numbers positive

Arithmetic shifts maintain the sign bit

- Used for signed numbers

Also rotate operations



12 © Cadence Design Systems, Inc. All rights reserved.

cadence

Simple multiplication or division by constant powers of 2 can be implemented with shift operations.

For example, $a*2$ can be implemented by shifting bit positions one place to the left and inserting a 0 in the LSB.

e.g. if $a = 101$ binary = 5 decimal, then $a*2 = 1010 = 10$ decimal.

Likewise a divide by a constant power of 2 is a right shift.

Logical shifts insert 0 at the LSB (left shift) or MSB (right shift). Therefore they can only be used with unsigned binary numbers, as inserting 0 in the MSB for a signed 2s complement number will make a negative number positive (remember MSB is the sign bit in 2s complement).

An arithmetic shift maintains the sign bit by recirculating the MSB for a right shift, and therefore is used for signed arithmetic.

There are also rotates, which recirculate bits. For rotates, data is never lost, just reordered. They are less useful in arithmetic.

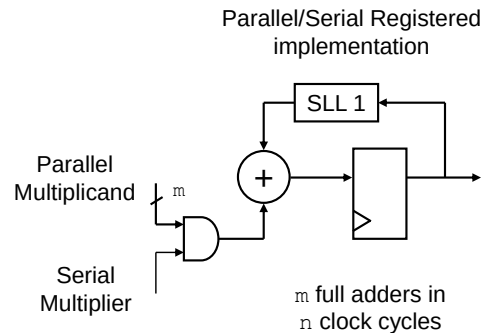
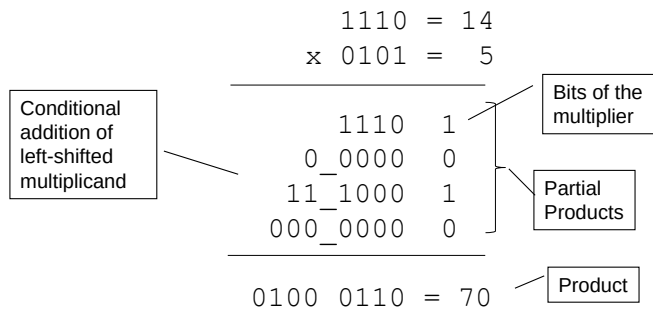
Binary Unsigned Multiplication

Multiplication is repeated, conditional addition (long multiplication)

- Easy in binary since digits are only 1 or 0

Multiplying an n bit multiplicand by m bit multiplier gives an $n+m$ product

Many different architectures depending on area/speed tradeoffs



13 © Cadence Design Systems, Inc. All rights reserved.

Multiplication is the conditional addition of a left-shifted multiplicand according to the bits of the multiplier. This is just like long multiplication taught in school. In binary, the calculation is easier than decimal as the bits are only 1 or 0, so the multiplicand is either added or not to the result.

In the diagram, the bold bits represent the bits added by the shift.

Multiplying an n bit multiplicand by an m bit multiplier requires $n+m$ bits for the product to hold the full width of the result without truncation.

From the calculation, we can see that multiplication can be implemented with an AND gate to check the multiplier bit and adders to sum the result, but there are many different implementations depending on area and speed requirements. The Parallel/Serial registered implementation is efficient in area – one AND and one adder – but is slow, taking m clock cycles to calculate the result.

Unsigned Multiplier Implementation With Shift Adder Array

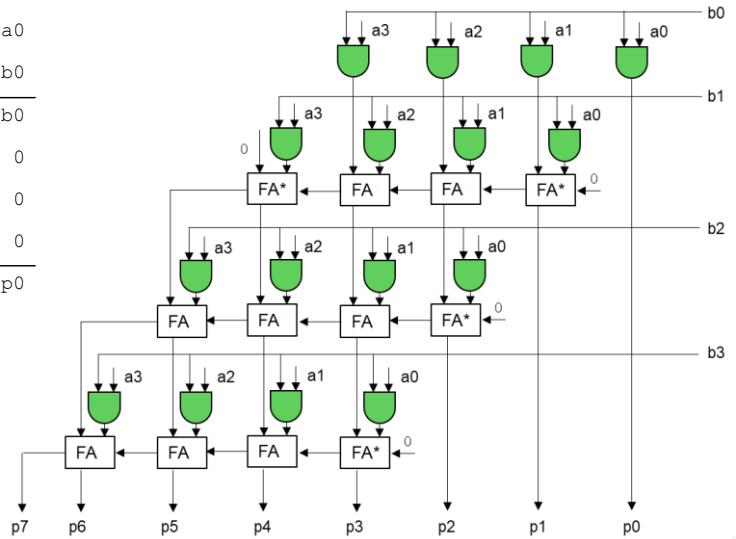
				a3	a2	a1	a0
				b3	b2	b1	b0
				a3b0	a2b0	a1b0	a0b0
			a3b1	a2b1	a1b1	a0b1	0
		a3b2	a2b2	a1b2	a0b2	0	0
	a3b3	a2b3	a1b3	a0b3	0	0	0
p7	p6	p5	p4	p3	p2	p1	p0

N bit multiply uses N^2 AND and (N^2-N) Full Adders (FA)

Techniques for further optimization

- Wallace tree, Dadda tree

Stages can be pipelined for better throughput



*can be replaced by a Half Adder

This page does not contain notes.

Signed Multiplication

Many methods to multiply signed (2's complement) numbers

1. Use unsigned multiplication
 - Derive sign of product from signs of operands
 $\text{sign_product} = \text{sign_a} \text{ xor } \text{sign_b}$
 - Make operands positive and multiply as unsigned
 - If product sign is negative, convert result

2. Modified shift adder array

3. Algorithmic

- e.g. Booth's algorithm, which relies on noting:

$$11011_2 = -00001_2 + 00100_2 - 01000_2$$

$$-5_{10} = -1_{10} + 4_{10} - 8_{10}$$

So

$$9 * -5 = 9 * (-1 + 4 - 8) = -9 + 36 - 72 = -45$$

- We have 3 partial products rather than the 4 of a shift-adder array



Signed multiplication, with 2's complement operands, requires a different architecture.

The simplest implementation of signed multiplication is to convert the operation to unsigned:

- Find the sign of the product by XORing the sign bits of the operands
- Convert any negative operands to positive
- Perform unsigned multiplication on the operands
- Convert the result to negative if required.

Simple Signed Multiplication

	a3	a2	a1	a0				
	b3	b2	b1	b0				
+	a3b0	a3b0	a3b0	a3b0	a3b0	a2b0	a1b0	a0b0
+	a3b1	a3b1	a3b1	a3b1	a2b1	a1b1	a0b1	0
+	a3b2	a3b2	a3b2	a2b2	a1b2	a0b2	0	0
-	a3b3	a3b3	a2b3	a1b3	a0b3	0	0	0
	p7	p6	p5	p4	p3	p2	p1	p0

Final partial product
is subtracted
(add 2's complement)

$$\begin{array}{r} 0101 = 5 \\ \times 1110 = -2 \\ \hline \end{array}$$

$$\begin{array}{r} 0000_0000 \\ 0000_1010 \\ 0001_0100 \\ 1101_1000 \\ \hline 1111_0110 = -10 \end{array}$$

Shift adder multiplier can also be used for signed (2's complement) values

However, to maintain sign bit:

- Partial products must be sign-extended
- Final partial product must be subtracted rather than added (or add 2s complement)



With some minor changes, the shift-adder array for unsigned multiplication can be used with signed numbers.

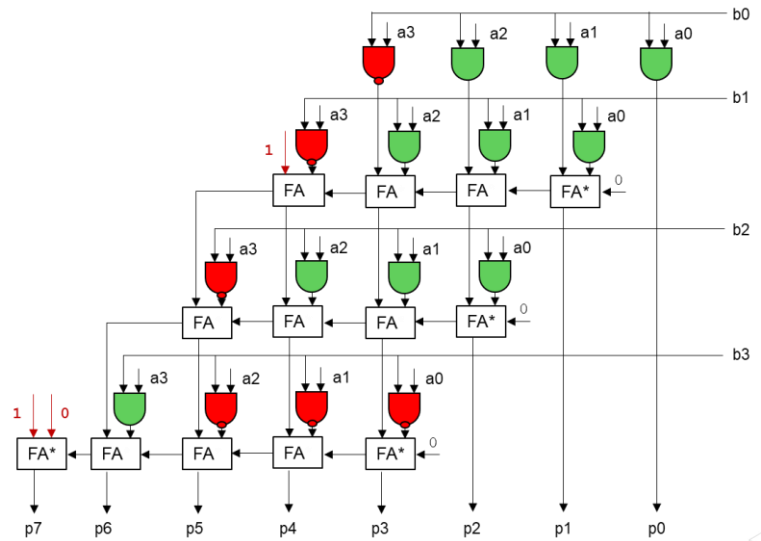
1. The partial products must be sign-extended to the width of the product.
2. The final partial product is subtracted rather than added to the product to maintain the sign bit.

Optimized Signed Multiplication Shift Array

			a3	a2	a1	a0	
			b3	b2	b1	b0	
+			a3b0	a2b0	a1b0	a0b0	
+			a3b1	a2b1	a1b1	a0b1	0
+			a3b2	a2b2	a1b2	a0b2	0
+		a3b3	a2b3	a1b3	a0b3	0	0
+	1	0	0	1	0	0	0
	p7	p6	p5	p4	p3	p2	p1

For 4 bit signed multiplication:

- 10 AND
- 6 NAND
- 13 Full Adders
 - Or 4 Half Adders, 9 Full Adders



*can be replaced by a Half Adder

This is the implementation of 4 bit signed multiplication using the shift-adder array. Compared to the unsigned multiplier, the changes are minimal:

- NAND gates (in red) replace selected AND gates
- We add 1 into the row 2 adder for p4
- For p7, we add 1 to the carry from p6

Remember a signed multiplier can be used for **both** signed and unsigned operands.

Booth's Recoding for Multiplication

From LSB, a string of 1s in the multiplier can be replaced with:

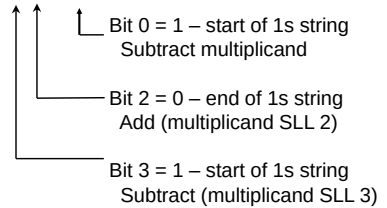
- A subtraction for the bit *beginning* the string
 - Including if the LSB is 1
- An addition for the bit *after* the end of the string

Consecutive 0s or 1s do not change the partial product

The multiplicand must be shifted according to the bit weighting

$$11011_2 = -00001_2 + 00100_2 - 01000_2$$

$$-5 = 11011$$



Works for signed, unsigned and any vector length:

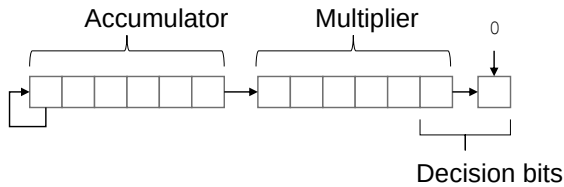
$$\begin{aligned} n * 11101110_2 &= -(n * 2^1) + (n * 2^4) - (n * 2^5) \\ &= -(n \ll 1) + (n \ll 4) - (n \ll 5) \\ n * -18_{10} &= -(n * 2_{10}) + (n * 16_{10}) - (n * 32_{10}) \end{aligned}$$

Shifted, 2's complement

$$\begin{array}{r} 01001 = 9 \\ 11011 = -5 \\ \hline 1111110111 \\ 0000100100 \\ 1110111000 \\ \hline 1111010011 = -45 \end{array}$$

This page does not contain notes.

Booth's Implementation



$m \times n$ multiplication uses an $(m+n+1)$ shift register, initialized as follows:

- Clear LSB to 0
- Load multiplier into next n bits
- Clear top m bits (accumulator)

Check the 2 LSBs and Add to accumulator as follows:

- 01 – Add multiplicand
- 10 – Add 2's complement multiplicand (subtract)
- 00, 11 – Do nothing

Shift Right Arithmetic (SRA) after each check

$$9_{10} * -5_{10} = 01001_2 * 11011_2 = 1111010011_2$$

$$-9_{10} = 10111_2$$

Load	0	0	0	0	0	1	1	0	1	1	0
Add (-m)	1	0	1	1	1	1	1	0	1	1	0
SRA ₁	1	1	0	1	1	1	1	1	0	1	1
SRA ₂	1	1	1	0	1	1	1	1	1	0	1
Add m	0	1	0	0	1						
	0	0	1	1	0	1	1	1	1	0	1
SRA ₃	0	0	0	1	1	0	1	1	1	1	0
Add (-m)	1	0	1	1	1						
	1	1	0	1	0	0	1	1	1	1	0
SRA ₄	1	1	1	0	1	0	0	1	1	1	1
SRA ₅	1	1	1	1	0	1	0	0	1	1	1
Product											

To multiply m and n bit numbers, one implementation of Booth's algorithm uses a $m+n+1$ shift register. The first m bits are the accumulator, the next n bits will initially contain the multiplier and the final LSB is initialised to 0.

The 2 register LSBs determine the operation for the current cycle. 01 signifies the end of a string of 1's and we add the multiplicand to the accumulator. 10 signifies the start of a string of 1's and we subtract the multiplicand (or add the 2's complement). For 00 and 11, we do nothing. After every check, we Shift Right Arithmetic the whole register, i.e. the MSB sign bit is maintained. Total number of shifts is equal to the number of multiplier bits n .

For $9 * -5$, we clear the accumulator and LSB, and load -5 (11011) into the multiplier.

The check bits are 10, so we add the 2's complement of 9 into the accumulator and SRA.

Next check bits are 11 so no addition, and we SRA.

Check bits are 01 so we add 9 (01001) to the accumulator and SRA.

Check bits are 10, so we add the 2's complement of 9 into the accumulator and SRA.

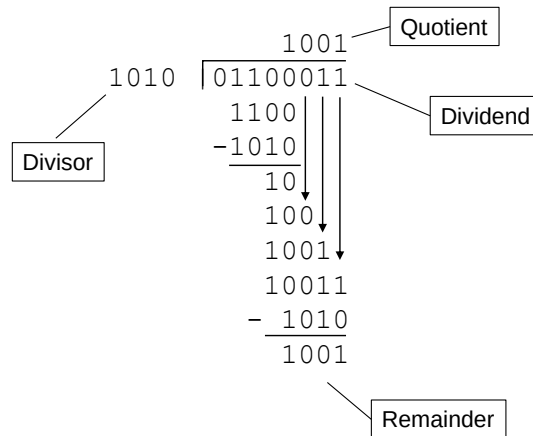
Check bits are 11 so no addition and we perform the final SRA.

The product is the top $m+n$ bits of the register.

Binary Division

Multiplication is repeated, conditional subtraction (long division)

- Easy in binary since digits are only 1 or 0



21 © Cadence Design Systems, Inc. All rights reserved.



Long division.

The example divides 99 (01100011) by 10 (1010).

Steps are:

1. Work from the left-hand side of the dividend and collate enough bits until we have a value greater than the divisor, i.e. 1100. Since $1100 > 1010$, the corresponding bit of the quotient is 1, and we subtract 1010 from 1100 to give 10.
2. We pull down the next bit of the dividend to make 100 and compare to the divisor. $100 < 1010$, so the corresponding quotient bit is 0.
3. We pull down the next bit of the dividend to make 1001 and compare to the divisor. $100 < 1010$, so the corresponding quotient bit is 0.
4. We pull down the final bit of the dividend to make 10011 and compare to the divisor. $10011 > 1010$, so the corresponding quotient bit is 1, and we subtract 1010 from 10011 to leave 1001.
5. As there are no more bits of the dividend to pull down, the divide is complete. The quotient is 1001 (9) and the remainder is 1001 (9).

So this can be implemented with repeated, conditional subtraction.

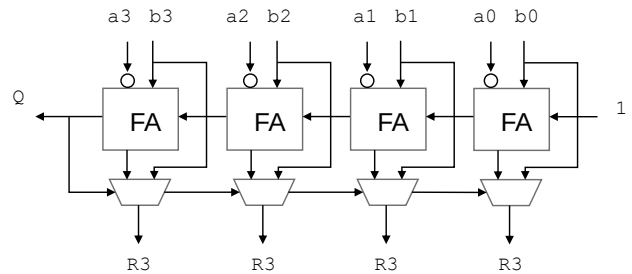
Binary Division Implementation

Division can be implemented with a shift subtract array

- Similar to multiplication...
- but (e.g.) using subtract/restore blocks

Subtract/Restore:

- Subtract a from b to give quotient (Q) and remainder (R)
 - If $b > a$, $Q = 1$ and selects $R = (b-a)$ (subtract)
 - If $b < a$, $Q = 0$ and selects $R = b$ (restore)



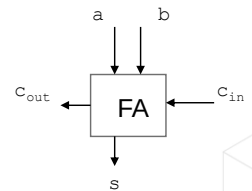
Examples:

$$\begin{array}{r} 0111 \quad 7 \\ + 1101 \quad -3 \\ \hline 10100 \end{array}$$

$Q=1$ selects $R = 0100$

$$\begin{array}{r} 0011 \quad 3 \\ + 1001 \quad -7 \\ \hline 01100 \end{array}$$

$Q=0$ selects $R = 0011$



We can use a similar architecture to the multiplication shift-adder array to implement division, but using subtractors instead of adders (or addition of 2's complement).

In the long division, we check if the divisor is greater than the selected bits of the dividend. If the divisor is greater, then the quotient bit is one and we subtract the divisor. If not, then the quotient is 0 and the dividend bits are unchanged.

Instead of testing the divisor/dividend before the subtraction, we can subtract first and use the result to determine if, for the next stage, we use the subtraction result (and $Q = 1$) or the unchanged dividend bits (and $Q = 0$).

We can use the carry-out of the full adder chain as both the quotient bit and to select between the subtraction result or the unchanged dividend for the next stage.

So a combination of the adder and a mux to pick the result is sometimes known as a subtract-restore block.

This simple single stage works where the divisor and dividend lengths are the same, but for longer dividends, we need a shift array architecture.

5-by-3 Bit Division With Subtract-Restore Array

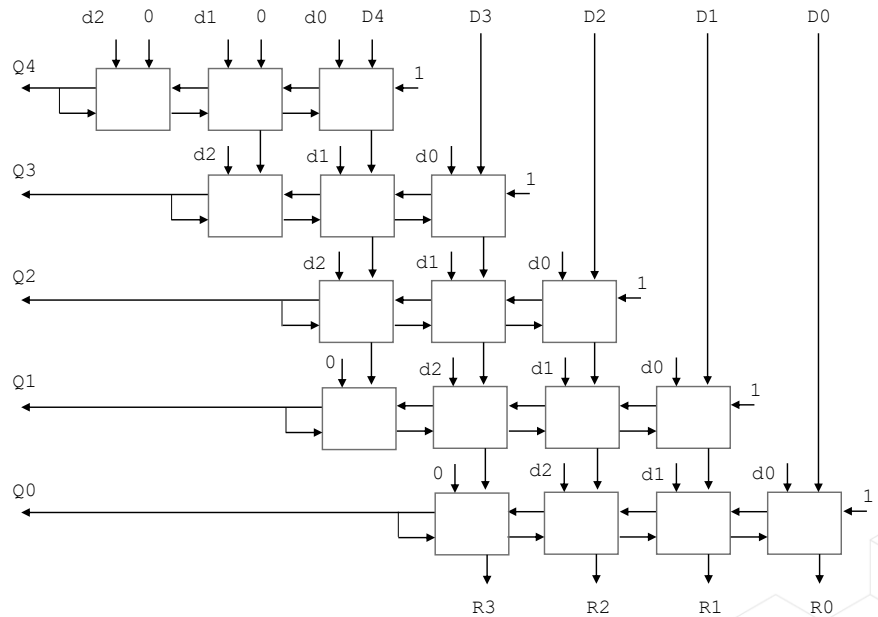
17 Subtract-Restore blocks

- Divides 5-bit dividend D
- By 3-bit divisor d
- Giving 5-bit Quotient Q
- and 4-bit Remainder R

Divides can be slow and expensive

Alternatives?

- Fixed point multiplication of $1/d$
- Approximations
- Number series



23 © Cadence Design Systems, Inc. All rights reserved.

cadence

Each block in the array is a subtract-restore block from the previous slide, i.e. one Full Adder (with one inverted input) and a mux.

Dividers can be slow and expensive in terms of area. For example, several basic processors do not contain divide logic at all. If you want a divide operation D/d , you need to calculate the reciprocal of the divisor d in fixed point binary and then use a multiplier to calculate $D * 1/d$.

Algorithmic division can use the same operation. e.g. Using Newton-Raphson to quickly find the reciprocal of d and multiply by D .

When you are performing division by constant values, we can either hardwire the divisor to the constant and let the synthesis tool optimize the logic, or build our own implementation using (for example) approximations or number series, depending on the constant value.

Approximations

Accuracy can be traded for speed and simplicity

- e.g. divide by 3

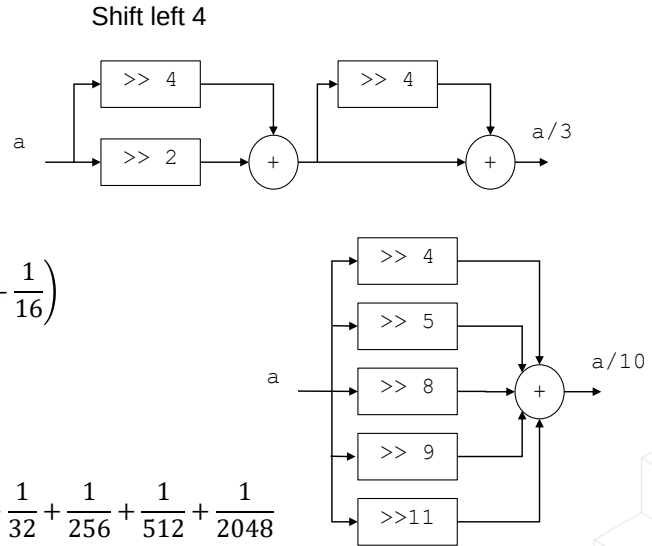
$$\frac{1}{3} \approx \frac{85}{256} = 0.332$$

$$\frac{85}{256} = \frac{5 \times 17}{16 \times 16} = \frac{(1+4) \times (16+1)}{16 \times 16} = \left(\frac{1}{16} + \frac{1}{4}\right) \times \left(1 + \frac{1}{16}\right)$$

- e.g. divide by 10

$$\frac{1}{10} \approx \frac{205}{2048} = 0.1001 = \frac{128 + 64 + 8 + 4 + 1}{2048} = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{512} + \frac{1}{2048}$$

Pick number of terms according to width of dividend



24 © Cadence Design Systems, Inc. All rights reserved.

cadence

Since constant shifts are effectively free in hardware (just a reordering of the bus) this means a divide by 3 can be implemented in just two adders.

Other approximations of varying accuracy, can be derived using similar methods...

E.g. A simpler divide by 10 for smaller numbers:

$$1/10 \approx 26/256 (0.1015625)$$

$$= (16 + 8 + 2)/256$$

$$= 1/16 + 1/32 + 1/128$$

An approximation with a fixed number of shifts needs to be carefully matched to the width of the dividend. Inaccuracy increases with the size of the dividend.

A number series allows us to select the number of shifts according to the width of the dividend for better accuracy (see next slide).

Number Series

Given:

$$\frac{1}{(1+x)} = (1 - x + x^2 - x^3 + x^4 + \dots)$$

Then:

$$\begin{aligned} \frac{1}{5} &= \frac{1}{4+1} = \frac{1}{4} \left(\frac{1}{1+\frac{1}{4}} \right) = 2^{-2} \left(\frac{1}{1+2^{-2}} \right) = 2^{-2} \times (1 - 2^{-2} + 2^{-4} - 2^{-6} + 2^{-8} - \dots) \\ &= (2^{-2} - 2^{-4} + 2^{-6} - 2^{-8} + 2^{-10} - \dots) \end{aligned}$$

$$\frac{a}{5} = \frac{a}{4} - \frac{a}{16} + \frac{a}{64} - \frac{a}{256} + \dots = (a \gg 2) - (a \gg 4) + (a \gg 6) - (a \gg 8) + \dots$$

Convergent number series can be used for control over accuracy

- Pick number of terms according to width of dividend
 - e.g. 8 bit vector could use just the first 3 terms
- Or could right-shift before and left-shift after division to accumulate bits normally truncated

25

© Cadence Design Systems, Inc. All rights reserved.



Number series allows control over the accuracy of results in, hopefully, a simple implementation using a shift and add architecture.

Not all implementations are as simple as $1/5$!

Greater accuracy for smaller numbers can be obtained by scaling the operand before the division operation,

- e.g. $1/16 * (16 * A) / 5$
- Multiply by 16 (Shift-Left 4)
- Divide by 5 as above
- Divide result by 16 to re-scale (Shift-right 4)

This allows some bits normally lost by left-shifting to be still considered in the result.

Summary

Synthesis tools do not always infer suitable implementations

- Complex operations
- Signed multiplication
- Division
- Even for simple operations on limited resources

Operations must be "designed" to fit available resources

- Area, timing, speed, power

Various implementation methods can help

- Algorithms
- Approximations

There are time/area trade-offs for each approach



For simple operations, a synthesis tool can infer arithmetical operations from code and select an architecture from time/area constraints.

For more complex operations, the structure and architecture of the operation may need to be built by the designer.

Exercise 1 : Thermometer Device

Aim:

- To convert a 9 bit signed temperature value (in degrees Celsius) to a 3 digit Binary Coded Decimal (BCD) plus sign format suitable for display on a 4 digit 7-segment LCD display.

Background:

- Input temperature range is +125 to -55oC

Procedure:

- What steps could we take to perform the data conversion?



This page does not contain notes.

Exercise 2 : Temperature Conversion

Aim:

- To take an 8 bit signed temperature value in degrees Celsius (–128 to +127 °C) and convert it to Fahrenheit
- The result should be an N bit signed number long enough to accommodate the full width of the conversion and rounded to the nearest °F

Background:

- To convert centigrade (C) to Fahrenheit (F),
 - $F = C * 9/5 + 32$

Procedure:

- This implies a multiplication and a division
- What are suitable arithmetic steps to perform the conversion which exploit the simplicity of the constants involved?



This page does not contain notes.